

第 1 章 Hibernate_day04

今日任务

- 案例一:使用 Hibernate 完成对客户条件查询
- 案例二:使用 Hibernate 完成对客户查询的优化

教学导航

教学目标	
教学方法	案例驱动法

案例一：完成 CRM 的客户的条件查询

1.1 案例需求：

1.1.1 需求描述

在 CRM 中我们需要对客户进行综合条件查询,效果如下:

当前位置: 客户管理 > 客户列表

客户名称: 客户来源: 客户级别: 筛选

客户名称	客户级别	客户来源	联系人	电话	手机	操作
王五	VIP客户	网络营销	李粉书	010-87659823	15726609865	修改 删除
李四	VIP客户	网络营销	张粉书	010-87659823	15726609865	修改 删除
小王	普通客户	电话营销	李粉书	010-87659823	15726609865	修改 删除

共[18]条记录, [6]页, 每页显示 3 条 [前一页] [后一页] 到 页 Go

那么我们如何完成组合条件查询的工作呢? 接下来我们需要来学习 Hibernate 中的检索方式就可以解决这类问题了。

1.2 相关知识

1.2.1 Hibernate 的检索方式:

在实际开发项目时, 对数据进行最多的操作就是查询, 数据的查询在所有 ORM 框架中都占有



极其重要的地位。那么，如何利用 Hibernate 查询数据呢？我们接下来就来学习 Hibernate 的检索方式。

1.2.1.1 Hibernate 检索方式的分类：

Hibernate 的检索方式主要有 5 种，分别为导航对象图检索方式、OID 检索方式、HQL 检索方式、QBC 检索方式和 SQL 检索方式。下面就对这 5 种检索方式的使用进行讲解。

1.2.2 对象图导航检索

对象图导航检索方式是根据已经加载的对象，导航到他的关联对象。它利用类与类之间的关系来检索对象。譬如要查找一个联系人对应的客户，就可以由联系人对象自动导航找到联系人所属的客户对象。当然，前提是必须在对象关系映射文件上配置了多对一的关系。其检索方式如下所示。

```
LinkMan linkMan = (LinkMan)session.get(LinkMan.class,11);  
Customer customer = linkMan.getCustomer();
```

1.2.3 OID 检索方式

OID 检索方式主要指用 Session 的 get()和 load()方法加载某条记录对应的对象。如下面两种加载客户对象的方式，就是 OID 检索方式，具体如下。

```
Customer customer = (Customer )session.get(Customer.class,1);  
Customer customer = (Customer )session.load(Customer.class,1);
```

1.2.4 HQL 检索

HQL (Hibernate Query Language) 是面向对象的查询语言，它和 SQL 查询语言有些相似，但它使用的是类、对象和属性的概念，而没有表和字段的概念。在 Hibernate 提供的各种检索方式中，HQL 是官方推荐的查询语言，也是使用最广泛的一种检索方式。它具有如下功能。

- 在查询语句中设定各种查询条件。
- 支持投影查询，即仅检索出对象的部分属性。
- 支持分页查询。
- 支持分组查询，允许使用 group by 和 having 关键字。
- 提供内置聚集函数，如 sum()、min()和 max()。
- 能够调用用户定义的 SQL 函数。
- 支持子查询，即嵌套查询。
- 支持动态绑定参数。

Hibernate 提供的 Query 接口是专门的 HQL 查询接口，它能够执行各种复杂的 HQL 查询语句。完整的 HQL 语句结构如下。

```
select...from...where...group by...having...order by...asc/desc
```

可见 HQL 查询非常类似于标准 SQL 查询。通常情况下，当检索数据表中的所有记录时，查询



语句中可以省略 select 关键字，示例如下所示。

```
String hql = "from Customer";
```

如果执行该查询语句，则会返回应用程序中的所有 Customer 对象，需要注意的是 Customer 是类名，而不是表名，类名需要区分大小写，而关键字 from 不区分大小写。

我们已经对 HQL 有了基本的了解，那么我们具体的来使用一下 HQL。

1.2.4.1 基本检索

```
@Test
/**
 * HQL: 基本检索
 */
public void demo2(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 基本查询
    /*Query query = session.createQuery("from Customer");
    List<Customer> list = query.list();*/
    // 起别名
    /*Query query = session.createQuery("from Customer c");
    Query query = session.createQuery("select c from Customer c");
    List<Customer> list = query.list();

    for (Customer customer : list) {
        System.out.println(customer);
    }

    tx.commit();
}*/
```

1.2.4.2 排序检索

```
@Test
/**
 * HQL: 排序检索
 */
public void demo3(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 基本查询
```



```
Query query = session.createQuery("from Customer order by cust_id desc");
List<Customer> list = query.list();

for (Customer customer : list) {
    System.out.println(customer);
}

tx.commit();
}
```

1.2.4.3 条件检索

```
@Test
/**
 * HQL: 条件查询
 */
public void demo4(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 按位置绑定参数
    /*Query query = session.createQuery("from Customer where cust_name = ?");
    // query.setString(0, "小军军");
    query.setParameter(0, "小军军");
    List<Customer> list = query.list();*/

    // 按名称绑定参数
    Query query = session.createQuery("from Customer where cust_name = :name");
    query.setParameter("name", "小添添");
    /*List<Customer> list = query.list();
    for (Customer customer : list) {
        System.out.println(customer);
    }*/
    Customer customer = (Customer) query.uniqueResult();
    System.out.println(customer);

    tx.commit();
}
```

1.2.4.4 分页检索

```
@Test
```




```
/**
 * 分页查询
 */
public void demo5(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    Query query = session.createQuery("from LinkMan order by lkm_id desc");
    query.setFirstResult(5);
    query.setMaxResults(5);
    List<LinkMan> list = query.list();
    for (LinkMan linkMan : list) {
        System.out.println(linkMan);
    }

    tx.commit();
}
```

1.2.4.5 统计检索

```
@Test
/**
 * 统计查询
 */
public void demo6(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    Query query = session.createQuery("select count(*) from Customer");
    Long num = (Long) query.uniqueResult();
    System.out.println(num);

    tx.commit();
}
```

1.2.4.6 投影检索

```
@Test
/**
 * 投影查询
 */
public void demo7(){
```



```
Session session = HibernateUtils.getCurrentSession();
Transaction tx = session.beginTransaction();
// 投影查询一行
/*List<String> list = session.createQuery("select cust_name from
Customer").list();
for (String string : list) {
    System.out.println(string);
}*/
// 投影查询多列:
/*List<Object[]> list = session.createQuery("select cust_id,cust_name from
Customer").list();
for (Object[] objects : list) {
    System.out.println(Arrays.toString(objects));
}*/

// 投影的构造的方式查询:
List<Customer> list = session.createQuery("select new
Customer(cust_id,cust_name) from Customer").list();
for (Customer customer : list) {
    System.out.println(customer);
}
tx.commit();
}
```

1.2.5 QBC 检索

QBC(Query By Criteria)是 Hibernate 提供的另一种检索对象的方式，它主要由 Criteria 接口、Criterion 接口和 Expression 类组成。Criteria 接口是 Hibernate API 中的一个查询接口，它需要由 session 进行创建。Criterion 是 Criteria 的查询条件，在 Criteria 中提供了 add(Criterion criterion)方法来添加查询条件。使用 QBC 检索对象的示例代码，如下所示。

```
//创建 criteria 对象
Criteria criteria = session.createCriteria(Customer.class);
//设定查询条件
Criterion criterion = Restrictions.eq("id", 1);
//添加查询条件
criteria.add(criterion);
//执行查询,返回查询结果
List<Customer> cs = criteria.list();
```

上述代码中查询的是 id 为 1 的 Customer 对象。

QBC 检索是使用 Restrictions 对象编写查询条件的，在 Restrictions 类中提供了大量的静态方法来创建查询条件。其常用的方法如表所示。

Restrictions 类提供的方法



方法名	说明
Restrictions.eq	等于
Restrictions.allEq	使用 Map，使用 key/value 进行多个等于的比较
Restrictions.gt	大于>
Restrictions.ge	大于等于>=
Restrictions.lt	小于
Restrictions.le	小于等于<=
Restrictions.between	对应 SQL 的 between 子句
Restrictions.like	对应 SQL 的 like 子句
Restrictions.in	对应 SQL 的 IN 子句
Restrictions.and	and 关系
Restrictions.or	or 关系
Restrictions.sqlRestriction	SQL 限定查询

对 QBC 查询有了一定的基础以后，我们具体来使用一下 QBC 完成查询。

1.2.5.1 基本检索

```
@Test
/**
 * 简单的查询
 */
public void demo1(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();
    // 简单查询:
    Criteria criteria = session.createCriteria(Customer.class);
    List<Customer> list = criteria.list();

    for (Customer customer : list) {
        System.out.println(customer);
    }
    tx.commit();
}
```

1.2.5.2 条件检索

```
@Test
/**
 * 条件查询
 */
public void demo2(){
```



```
Session session = HibernateUtils.getCurrentSession();
Transaction tx = session.beginTransaction();
// 条件查询:
Criteria criteria = session.createCriteria(Customer.class);
// 设置条件:
// criteria.add(Restrictions.eq("cust_name", "小军军"));
criteria.add(Restrictions.like("cust_name", "%小%"));
criteria.add(Restrictions.gt("cust_id", 11));

List<Customer> list = criteria.list();

for (Customer customer : list) {
    System.out.println(customer);
}
tx.commit();
}
```

1.2.5.3 分页检索

```
@Test
/**
 * 分页查询
 */
public void demo3(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();
    // 条件查询:
    Criteria criteria = session.createCriteria(LinkMan.class);
    // 设置分页查询:
    criteria.setFirstResult(5);
    criteria.setMaxResults(5);

    List<LinkMan> list = criteria.list();

    for (LinkMan linkMan : list) {
        System.out.println(linkMan);
    }
    tx.commit();
}
```



1.2.5.4 排序检索

```
@Test
/**
 * 排序查询
 */
public void demo4(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 排序查询:
    Criteria criteria = session.createCriteria(LinkMan.class);
    criteria.addOrder(Order.desc("lkm_id"));

    List<LinkMan> list = criteria.list();

    for (LinkMan linkMan : list) {
        System.out.println(linkMan);
    }

    tx.commit();
}
```

1.2.5.5 统计检索

```
@Test
// 统计查询:
public void demo4(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    Criteria criteria = session.createCriteria(LinkMan.class);
    criteria.setProjection(Projections.rowCount());
    Long count = (Long) criteria.uniqueResult();

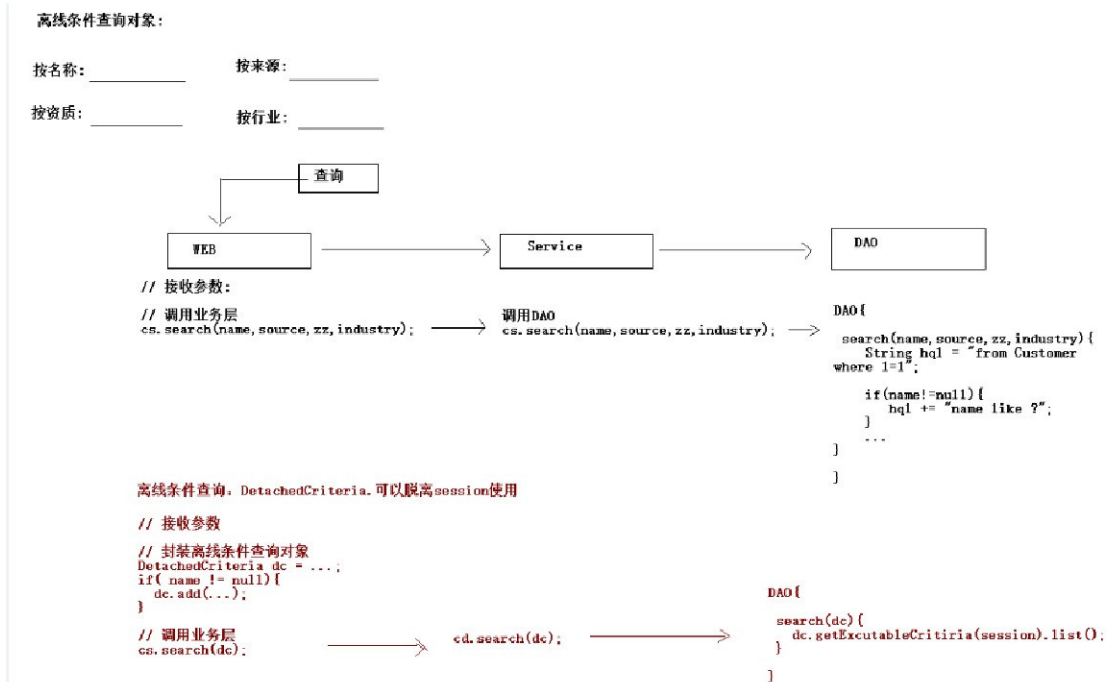
    System.out.println(count);
    tx.commit();
}
```




1.2.5.6 离线条件检索

DetachedCriteria 翻译为离线条件查询，因为它是可以脱离 Session 来使用的一种条件查询对象，我们都知道 Criteria 对象必须由 Session 对象来创建。那么也就是说必须先有 Session 才可以生成 Criteria 对象。而 DetachedCriteria 对象可以在其他层对条件进行封装。

这个对象也是比较有用的，尤其在 SSH 整合以后这个对象经常会使用。它的主要优点是做一些特别复杂的条件查询的时候，往往会在 WEB 层向业务层传递很多的参数，业务层又会将这些参数传递给 DAO 层。最后在 DAO 中拼接 SQL 完成查询。有了离线条件查询对象后，那么这些工作都可以不用关心了，我们可以在 WEB 层将数据封装好，传递到业务层，再由业务层传递给 DAO 完成查询。



我们可以先简单的测试一下离线条件查询对象，然后具体的使用我们会在后期整合中使用。到那时会更加体会出它的优势。

```
@Test
/**
 * 离线条件查询:DetachedCriteria(SSh 整合经常使用.)
 * * 可以脱离 session 设置参数.
 */
public void demo6(){
    // 获得一个离线条件查询的对象
    DetachedCriteria detachedCriteria =
    DetachedCriteria.forClass(Customer.class);

    detachedCriteria.add(Restrictions.eq("cust_name", "小童童"));

    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();
```



```

        List<Customer> list =
detachedCriteria.getExecutableCriteria(session).list();// 离线条件查询对象与 session 绑定.

        for (Customer customer : list) {
            System.out.println(customer);
        }

        tx.commit();
    }

```

1.2.6 本地 SQL 检索方式

采用 HQL 或 QBC 检索方式时，Hibernate 生成标准的 SQL 查询语句，适用于所有的数据库平台，因此这两种检索方式都是跨平台的。但有的应用程序可能需要根据底层数据库的 SQL 方言，来生成一些特殊的查询语句。在这种情况下，可以利用 Hibernate 提供的 SQL 检索方式。使用 SQL 检索方式检索对象的示例代码，如下所示：

```

SQLQuery sqlQuery = session.createSQLQuery("select id,name,age,city from
customer");

```

在这，SQL 的检索方式我们不做太多的介绍。之前无论我们使用的是 HQL 或是 QBC 或是 SQL 其实都是单表的查询，而实际的开发中我们往往需要多个表的联合查询才可以获得我们想要的数据库。那么接下来我们就来学习一下多表的查询吧。

1.2.7 Hibernate 的多表查询

在做多表查询之前，我们需要先来回顾下使用 SQL 是如何完成多表的查询的。在学习 SQL 语句的时候进行多表联合查询一般都会采用连接查询，那么我们就来回顾一下 SQL 中的多表的联合查询。

1.2.7.1 SQL 多表联合查询:

- 连接查询:

- 交叉连接

交叉连接返回的结果是被连接的两个表中所有数据行的笛卡尔积，也就是返回第一个表中符合查询条件的数据行数乘以第二个表中符合查询条件的数据行数，例如 department 表中有 4 个部门，employee 表中有 4 个员工，那么交叉连接的结果就有 4*4=16 条数据。

交叉连接的语法格式如下：

```

SELECT * from 表1 CROSS JOIN 表2;

```

也可以写为如下格式：

```

SELECT * from 表1,表2;

```

从上述描述情况可以看出，交叉连接的结果就是两个表中所有数据的组合。需要注意的是，在实际开发中这种业务需求是很少见的，一般不会使用交叉连接，而是使用具体的条件对数据进行有目的的查询。

- 内连接



内连接（INNER JOIN）又称简单连接或自然连接，是一种常见的连接查询。内连接使用比较运算符对两个表中的数据进行比较，并列出与连接条件匹配的数据行，组合成新的记录，也就是说在内连接查询中，只有满足条件的记录才能出现在查询结果中。内连接查询的语法格式如下所示：

```
SELECT 查询字段 FROM 表1 [INNER] JOIN 表2 ON 表1.关系字段 = 表2.关系字段
```

在上述语法格式中，INNER JOIN 用于连接两个表，ON 来指定连接条件，其中 INNER 可以省略。内连接其实还可以细分为如下两类

- ◆ 隐式内连接：顾名思义隐式的就是我们看不到 inner join 的关键字。而使用 where 关键字替代。

```
SELECT * from 表1 ,表2 where 表1.关系字段 = 表2.关系字段;
```

- ◆ 显示内连接：显示的就是在语句中明显的调用了 inner join 的关键字。

```
SELECT * from 表1 inner join 表2 on 表1.关系字段 = 表2.关系字段;
```

```
SELECT * from 表1 join 表2 on 表1.关系字段 = 表2.关系字段;
```

■ 外连接

前面讲解的内连接查询中，返回的结果只包含符合查询条件和连接条件的数据，然而有时还需要包含没有关联的数据，即返回查询结果中不仅包含符合条件的数据，而且还包括左表（左连接或左外连接）、右表（右连接或右外连接）或两个表（全外连接）中的所有数据，此时就需要使用外连接查询，外连接分为左连接和右连接。

外连接的语法格式如下：

```
SELECT 所查字段 FROM 表1 LEFT|RIGHT [OUTER] JOIN 表2
```

```
ON 表1.关系字段 = 表2.关系字段 WHERE 条件
```

外连接的语法格式和内连接类似，只不过使用的是 LEFT JOIN、RIGHT JOIN 关键字，其中关键字左边的表被称为左表，关键字右边的表被称为右表。

在使用左连接和右连接查询时，查询结果是不一致的，具体如下：

- LEFT JOIN（左连接）：返回包括左表中的所有记录和右表中符合连接条件的记录。

```
SELECT * from 表1 left outer join 表2 on 表1.关系字段 = 表2.关系字段;
```

```
SELECT * from A left join 表2 on 表1.关系字段 = 表2.关系字段;
```

- RIGHT JOIN（右连接）：返回包括右表中的所有记录和左表中符合连接条件的记录。

```
SELECT * from 表1 right outer join 表2 on 表1.关系字段 = 表2.关系字段;
```

```
SELECT * from A right join 表2 on 表1.关系字段 = 表2.关系字段;
```

SQL 语句的连接查询我们会写了，那么 Hibernate 中的 HQL 如何进行连接查询呢？我们来看一下 Hibernate 的多表查询

1.2.7.2 HQL 连接查询

Hibernate 进行多表查询与 SQL 其实是很相似的，但是 HQL 会在原来 SQL 分类的基础上又多出来一些操作。

HQL 的多表连接查询的分类如下：

- 交叉连接

- 内连接

- ◆ 显示内连接

- ◆ 隐式内连接

- ◆ 迫切内连接



- 外连接
 - ◆ 左外连接
 - ◆ 迫切左外连接
 - ◆ 右外连接

其实这些连接查询语法大致都是一致的，就是 HQL 查询的是对象而 SQL 查询的是表。那么我们来比较一下 SQL 和 HQL 的连接查询。

SQL 连接查询：

```
SELECT * FROM cst_customer c INNER JOIN cst_linkman l ON c.cust_id = l.lkm_cust_id;
```

HQL 连接的查询：

```
from Customer c inner join c.linkMans
```

在 HQL 中，我们不用写关联字段了，因为客户中的联系人的集合其实对应的就是外键，所以我们在 inner join 的后面直接可以写 c.linkMans。在控制台输出的语句的格式如下：

```

Hibernate:
select
  customer0_.cust_id as cust_id1_0_,
  linkmans1_.lkm_id as lkm_id1_1_,
  customer0_.cust_name as cust_nam2_0_,
  customer0_.cust_source as cust_sou3_0_,
  customer0_.cust_industry as cust_ind4_0_,
  customer0_.cust_level as cust_lev5_0_,
  customer0_.cust_phone as cust_pho6_0_,
  customer0_.cust_mobile as cust_mob7_0_,
  linkmans1_.lkm_name as lkm_name2_1_1_,
  linkmans1_.lkm_gender as lkm_gend3_1_1_,
  linkmans1_.lkm_phone as lkm_phon4_1_1_,
  linkmans1_.lkm_mobile as lkm_mobi5_1_1_,
  linkmans1_.lkm_email as lkm_ema6_1_1_,
  linkmans1_.lkm_qq as lkm_qq7_1_1_,
  linkmans1_.lkm_position as lkm_posi8_1_1_,
  linkmans1_.lkm_memo as lkm_memo9_1_1_,
  linkmans1_.lkm_cust_id as lkm_cus10_1_1_
from
  cst_customer customer0_
inner join
  cst_linkman linkmans1_
    on customer0_.cust_id=linkmans1_.lkm_cust_id
  
```

我们发现如果这样写 HQL 语句的话，生成的底层的 SQL 语句就是使用 inner join 进行连接，而连接的条件就是 `customer0_.cust_id=linkmans1_.lkm_cust_id`。就是两个表的关联的字段。所以 HQL 的连接不用写具体的 on 条件。直接写关联的属性即可。

那么我们已经知道了 HQL 的内连接的具体的写法了，那么迫切内连接和内连接写法和使用上有什么区别呢？

迫切内连接其实就是在内连接的 inner join 后添加一个 fetch 关键字。我们来比对一下：
内连接：

```

@Test
/**
 * HQL 的多表的连接查询：内连接
 */
public void demo8() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    session.createQuery("from Customer c inner join c.linkMans").list();

    transaction.commit();
}
  
```

控制台输出效果如下：



```
Console  Tasks  JUnit
<terminated> HibernateDemo1.demo8 [JUnit] C:\Program Files (x86)\Java\jre7\bin\javaw.exe (2016年8月1日 上午10:33:18)
10:33:29,788  INFO QueryTranslatorFactoryInitiator:47 - HH000397: Using ASTQueryTranslatorFactory
Hibernate:
select
  customer0_.cust_id as cust_id1_0_0_,
  linkmans1_.lkm_id as lkm_id1_1_1_,
  customer0_.cust_name as cust_nam2_0_0_,
  customer0_.cust_source as cust_sou3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_lev5_0_0_,
  customer0_.cust_phone as cust_pho6_0_0_,
  customer0_.cust_mobile as cust_mob7_0_0_,
  linkmans1_.lkm_name as lkm_name2_1_1_,
  linkmans1_.lkm_gender as lkm_gend3_1_1_,
  linkmans1_.lkm_phone as lkm_phon4_1_1_,
  linkmans1_.lkm_mobile as lkm_mobi5_1_1_,
  linkmans1_.lkm_email as lkm_ema6_1_1_,
  linkmans1_.lkm_qq as lkm_qq7_1_1_,
  linkmans1_.lkm_position as lkm_posi8_1_1_,
  linkmans1_.lkm_memo as lkm_memo9_1_1_,
  linkmans1_.lkm_cust_id as lkm_cus10_1_1_
from
  cst_customer customer0_
inner join
  cst_linkman linkmans1_
    on customer0_.cust_id=linkmans1_.lkm_cust_id
```

迫切内连接:

```
@Test
/**
 * HQL 的多表的连接查询：内连接
 */
public void demo9() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    session.createQuery("from Customer c inner join fetch c.linkMans").list();

    transaction.commit();
}
```

控制台输出效果如下:

```
Console  Tasks  JUnit
<terminated> HibernateDemo1.demo8 [JUnit] C:\Program Files (x86)\Java\jre7\bin\javaw.exe (2016年8月1日 上午10:32:17)
10:32:31,012  INFO QueryTranslatorFactoryInitiator:47 - HH000397: Using ASTQueryTranslatorFactory
Hibernate:
select
  customer0_.cust_id as cust_id1_0_0_,
  linkmans1_.lkm_id as lkm_id1_1_1_,
  customer0_.cust_name as cust_nam2_0_0_,
  customer0_.cust_source as cust_sou3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_lev5_0_0_,
  customer0_.cust_phone as cust_pho6_0_0_,
  customer0_.cust_mobile as cust_mob7_0_0_,
  linkmans1_.lkm_name as lkm_name2_1_1_,
  linkmans1_.lkm_gender as lkm_gend3_1_1_,
  linkmans1_.lkm_phone as lkm_phon4_1_1_,
  linkmans1_.lkm_mobile as lkm_mobi5_1_1_,
  linkmans1_.lkm_email as lkm_ema6_1_1_,
  linkmans1_.lkm_qq as lkm_qq7_1_1_,
  linkmans1_.lkm_position as lkm_posi8_1_1_,
  linkmans1_.lkm_memo as lkm_memo9_1_1_,
  linkmans1_.lkm_cust_id as lkm_cus10_1_1_,
  linkmans1_.lkm_cust_id as lkm_cus10_1_0_,
  linkmans1_.lkm_id as lkm_id1_1_0_
from
  cst_customer customer0_
inner join
  cst_linkman linkmans1_
    on customer0_.cust_id=linkmans1_.lkm_cust_id
```

我们会发现无论是内连接或是迫切内连接发送的底层 SQL 都是一样的，而且在生成的 SQL 语句中也没有 fetch 关键字，当然 fetch 本身就不是 SQL 语句的关键字。所以一定要注意，fetch 只能在 HQL 中使用的，生成了 SQL 语句以后，fetch 就消失了。那么 fetch 到底有什么作用呢？



其实我们知道 HQL 内连接查询的和 SQL 的内连接查询到的结果集是一样的，都是两个表的交集部分的数据。

	cust_id1_0_0_	lkm_id1_1_1_	cust_nam2_0_0_	cust_sou3_0_0_	cust_ind4_0_0_	cust_lev5
☐	1	1	张总	网络推广	教育培训	经理
☐	1	2	张总	网络推广	教育培训	经理
☐	2	3	刘总	电话营销	房地产	经理

然后在封装数据的时候，普通内连接会将属于客户的数据封装到 Customer 对象中，会将属于联系人的数据封装到 LinkMan 对象中，所以每条记录都会是装有两个对象的集合，所以封装以后的数据是 List<Object[]>，在 Object[]中有两个对象一个是 Customer 另一个是 LinkMan。所以在使用普通内连接的时候可以如下编写代码：

```
@Test
/**
 * HQL 的多表的连接查询：内连接
 */
public void demo8(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    List<Object[]> list = session.createQuery("from Customer c inner join c.linkMans").list();

    for (Object[] objects : list) {
        System.out.println(Arrays.toString(objects));
    }

    transaction.commit();
}
```

那么加了 fetch 以后，虽然我们查询到的数据是一样的，但是 Hibernate 发现 HQL 中有 fetch 就会将数据封装到一个对象中，把属于客户的数据封装到 Customer 对象中，将属于联系人的部分封装到 Customer 中的联系人的集合中，这样最后封装完成以后是一个 List<Customer>中。所以使用迫切内连接的时候可以如下编写代码：

```
@Test
/**
 * HQL 的多表的连接查询：内连接
 */
public void demo8(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    List<Customer> list = session.createQuery("from Customer c inner join fetch c.linkMans").list();

    for (Customer customer : list) {
        System.out.println(customer);
    }
}
```




```
transaction.commit();  
}
```

其实内连接和迫切内连接的主要区别就在与封装数据，因为他们查询的结果集都是一样的，生成底层的 SQL 语句也是一样的。

- 内连接：发送就是内连接的语句，封装的时候将属于各自对象的数据封装到各自的对象中，最后得到一个 List<Object[]>。
- 迫切内连接：发送的是内连接的语句，需要在编写 HQL 的时候在 join 后添加一个 fetch 关键字，Hibernate 会发送 HQL 中的 fetch 关键字，从而将每条数据封装到对象中，最后得到一个 List<Customer>。

但是迫切内连接封装以后会出现重复的数据，因为我们查询到目前有三条记录，就会被封装到三个对象中，其实我们真正的客户对象只有两个所以往往自己在手动编写迫切内连接的时候会使用 distinct 去掉重复值。

```
@Test  
/**  
 * HQL 的多表的连接查询：内连接  
 */  
public void demo8(){  
    Session session = HibernateUtils.getCurrentSession();  
    Transaction transaction = session.beginTransaction();  
  
    List<Customer> list = session.createQuery("select distinct c from Customer  
c inner join fetch c.linkMans").list();  
  
    for (Customer customer : list) {  
        System.out.println(customer);  
    }  
  
    transaction.commit();  
}
```

到这我们已经对 Hibernate 查询已经学习完了，那么我们应该会使用 Hibernate 的这些查询应对今后的开发了。但是其实 Hibernate 的查询的效率并不高，我们需要对这些查询进行优化，才能达到高效的编程。

案例二:完成对 CRM 的查询优化

1.3 案例需求

1.3.1 需求描述

在 CRM 的案例中有很多地方需要用到查询操作，但是 Hibernate 本身的查询的效率不是很好，尤其是在获取关联对象的时候，那么我们需要对一些查询语句进行一些优化操作。才能更好的去提



升性能。如何去优化这些查询呢，我们需要来学习 Hibernate 的抓取策略。

1.4 相关知识

1.4.1 Hibernate 的抓取策略:

1.4.1.1 什么是抓取策略

抓取策略是当应用程序需要在(Hibernate 实体对象图的)关联关系间进行导航的时候，Hibernate 如何获取关联对象的策略。

Hibernate 的抓取策略是 Hibernate 提升性能的一种手段，可以在获取关联对象的时候，对发送的语句进行优化，但是往往抓取策略需要和延迟加载一起使用来提升性能。我们首先来学习一下延迟加载的相关内容吧。

在学习延迟加载内容之前，我们需要了解延迟加载的分类。

1.4.1.2 延迟加载的分类:

延迟加载在前面我们有过简单的介绍，延迟加载(lazy load)是(也称为懒加载)Hibernate 关联关系对象默认的加载方式，延迟加载机制是为了避免一些无谓的性能开销而提出来的，所谓延迟加载就是当在真正需要数据的时候，才真正执行数据加载操作。

通常将延迟加载分为两类：一类叫做类级别延迟，另一类叫做关联级别的延迟。类级别的延迟指的是查询某个对象的时候，是否采用有延迟，这个通常在<class>标签上配置 lazy 属性。关联级别的延迟指的是，查询一个对象的关联对象的时候是否采用延迟加载。这个通常在<set>或<many-to-one>上配置 lazy 属性。

【类级别的延迟加载】

使用 load 方法检索某个对象的时候，这个类是否采用延迟加载的策略，就是类级别的延迟。类级别的延迟一般在<class>上配置 lazy 属性，lazy 的默认值是 true。默认是延迟加载的，所以使用 load 方法去查询的时候，不会马上发送 SQL 语句，当真正使用该对象的时候，才会发送 SQL 语句。

```
Customer customer = session.load(Customer.class,11);
```

其实如果不想使用延迟加载也有很多种方法，当然最简单的就是将这个类的映射文件上的 lazy 设置为 false，当然也可以将这个持久化类改为 final 修饰，之前也介绍过，如果改为 final 修饰的话。就无法生成代理类，就会使延迟加载失效。

这是类级别的延迟加载，**类级别的延迟加载一般我们不进行修改，采用默认值 lazy="true"就可以了。**

其实主要的是关联级别的延迟加载，关联级别的延迟加载指的是查询到某个对象以后，检索它的关联对象的时候是否采用延迟加载。

【关联级别的延迟加载】

```
Customer customer = session.get(Customer.class,11);  
Set<LinkMan> linkMans = customer.getLinkMans();
```

通过客户查询其关联的联系人对象，在查询联系人的时候是否采用延迟加载称为是关联级别的延迟。关联级别的延迟通常是在<set>和<many-to-one>上来进行配置。



<set>标签上的 lazy 通常有三个取值：

- true：默认值，采用延迟加载
- false：检索关联对象的时候，不采用延迟加载。
- extra：及其懒惰的。

<many-to-one>标签上的 lazy 通常有三个取值：

- proxy：默认值，是否采用延迟取决于一的一方类上的 lazy 属性的值。
- false：检索关联对象的时候，不采用延迟加载。
- no-proxy：不用研究。

延迟加载介绍过了，延迟加载也是往往和抓取策略一起使用提升开发的程序的性能的。那么接下来我们要来研究抓取策略了。

抓取策略指的是查询到某个对象的时候，如何抓取其关联对象。这个也可以通过配置完成。在关联对象的标签上配置 fetch 属性。关联上就分为是在<set>和<many-to-one>上，也都有不同的取值。

<set>标签上的 fetch 通常有三个取值：

- select：默认值，发送的是普通的 select 语句查询。
- join：发送一条迫切左外连接去查询。
- subselect：发送一条子查询语句查询其关联对象。

<many-to-one>标签上的 fetch 有两个取值：

- select：默认值，发送一条普通的 select 语句查询关联对象。
- join：发送一条迫切左外连接语句查询其关联对象。

这样来看在 set 上配置 fetch 有三个值，lazy 有三个值，这样就会产生很多种的效果。其实不用担心，因为 fetch 如果设置为 join，lazy 就会失效了。

我们可以通过代码测试的方式来查看每种取值的效果。在<set>标签上的 fetch 和 lazy

```
@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。(默认情况)
 */
public void demo3(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 查询一号客户
    Customer customer = session.get(Customer.class, 1l); // 发送一条 select 语句
    查询 1 号客户

    // 获得一号客户的联系人的数量：
    System.out.println(customer.getLinkMans().size()); // 发送一条 select 语句查
    询 1 号客户的所有联系人。

    transaction.commit();
}
```




```
@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
 * 在<set>上配置 fetch="select" lazy="true"
 */
public void demo4() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 查询一号客户
    Customer customer = session.get(Customer.class, 11); // 发送一条 select 语句
    查询 1 号客户

    // 获得一号客户的联系人的数量：
    System.out.println(customer.getLinkMans().size()); // 发送一条 select 语句查
    询 1 号客户的所有联系人。

    transaction.commit();
}

@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
 * 在<set>上配置 fetch="select" lazy="false"
 */
public void demo5() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 查询一号客户
    Customer customer = session.get(Customer.class, 11); // 发送两条 select 语句 1
    条：查询 1 号客户 1 条：查询 1 号客户的所有联系人

    // 获得一号客户的联系人的数量：
    System.out.println(customer.getLinkMans().size()); // 不发送 sql 语句

    transaction.commit();
}

@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
 * 在<set>上配置 fetch="select" lazy="extra"
 */
public void demo6() {
    Session session = HibernateUtils.getCurrentSession();
```




```
Transaction transaction = session.beginTransaction();
// 查询一号客户
Customer customer = session.get(Customer.class, 1l); // 发送一条 select 语句
查询 1 号客户

// 获得一号客户的联系人的数量:
System.out.println(customer.getLinkMans().size()); // 发送一条 select
count(*) 语句统计个数。

transaction.commit();
}

@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
 * 在<set>上配置 fetch="join" lazy="失效"
 */
public void demo7(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 查询一号客户
    Customer customer = session.get(Customer.class, 1l); // 发送一条迫切左外连接
    直接将关联的对象一起查询了。

    // 获得一号客户的联系人的数量:
    System.out.println(customer.getLinkMans().size()); // 不发送了。

    transaction.commit();
}

@SuppressWarnings("unchecked")
@Test
/**
 * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
 * 在<set>上配置 fetch="subselect" lazy="true"
 */
public void demo8(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    // 子查询不能查询某一个对象, 查询多个对象来测试:
    List<Customer> list = session.createQuery("from Customer").list(); // 发送
    查询客户的语句
```



```

        for (Customer customer : list) {
            System.out.println(customer.getLinkMans().size()); // 发送一条子查询 查询 客户关联的联系人
        }

        transaction.commit();
    }

    @SuppressWarnings("unchecked")
    @Test
    /**
     * 查询 1 号客户，同时查询 1 号客户的联系人的数量。
     * 在<set>上配置 fetch="subselect" lazy="false"
     */
    public void demo9() {
        Session session = HibernateUtils.getCurrentSession();
        Transaction transaction = session.beginTransaction();

        // 子查询不能查询某一个对象，查询多个对象来测试：
        List<Customer> list = session.createQuery("from Customer").list(); // 发送查询所有客户的及子查询查询客户的联系人

        for (Customer customer : list) {
            System.out.println(customer.getLinkMans().size());
        }

        transaction.commit();
    }

```

在<set>标签上的抓取策略已经明白了，那么在<many-to-one>上的 fetch 和 lazy 又是什么样的效果呢？我们也可以通过程序来测试一下：

```

@Test
/**
 * 查询 1 号联系人，同时查询 1 号联系人所关联客户的信息（默认情况）
 */
public void demo1() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    // 查询 1 号联系人：
    LinkMan linkMan = session.get(LinkMan.class, 11); // 发送一条 select 语句查询 1 号联系人

    System.out.println(linkMan.getCustomer().getCust_name()); // 发送一条

```



select 语句查询 1 号联系人对应客户的信息

```
        transaction.commit();
    }

    @Test
    /**
     * 查询 1 号联系人，同时查询 1 号联系人所关联客户的信息
     * fetch="select" lazy="proxy"
     */
    public void demo2(){
        Session session = HibernateUtils.getCurrentSession();
        Transaction transaction = session.beginTransaction();

        // 查询 1 号联系人：
        LinkMan linkMan = session.get(LinkMan.class, 11); // 发送一条 select 语句查询
1 号联系人

        System.out.println(linkMan.getCustomer().getCust_name()); // 发 送 一 条
select 语句查询 1 号联系人对应客户的信息

        transaction.commit();
    }

    @Test
    /**
     * 查询 1 号联系人，同时查询 1 号联系人所关联客户的信息
     * fetch="select" lazy="false"
     */
    public void demo3(){
        Session session = HibernateUtils.getCurrentSession();
        Transaction transaction = session.beginTransaction();

        // 查询 1 号联系人：
        LinkMan linkMan = session.get(LinkMan.class, 11); // 发送一条 select 语句查询
1 号联系人 发送一条 select 语句查询 1 号联系人对应客户的信息

        System.out.println(linkMan.getCustomer().getCust_name()); // 不发送

        transaction.commit();
    }

    @Test
    /**
```



```

* 查询 1 号联系人，同时查询 1 号联系人所关联客户的信息
* fetch="join" lazy="失效"
*/

public void demo4() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    // 查询 1 号联系人：
    LinkMan linkMan = session.get(LinkMan.class, 11); // 发送一条迫切左外连接查询
    关联对象

    System.out.println(linkMan.getCustomer().getCust_name()); // 不发送

    transaction.commit();
}

```

1.4.1.3 <set>集合上的 fetch 和 lazy:

fetch: 控制的是查询其关联对象的时候采用的 SQL 语句的格式.

- * select :默认值. 发送一条 select 语句查询其关联对象.
- * join :发送 一条迫切左外连接查询关联对象.
- * subselect :发送 一条子查询查询其关联对象.

lazy: 控制的是查询其关联对象的时候是否采用延迟加载的策略.

- * true :默认值. 默认查询联系人的时候采用延迟加载.
- * false :查询关联对象的时候 不采用延迟加载.
- * extra :及其懒惰. 查询关联对象的时候 采用比延迟加载更懒惰的方式进行查询.

我们可以简单的总结一下 fetch 和 lazy 的作用，其实 fetch 主要控制抓取关联对象的时候的发送 SQL 语句的格式的。lazy 主要控制查询其关联对象的时候是否采用延迟加载的。

1.4.1.4 批量抓取

在抓取的策略中有一种叫做批量抓取，就是同时查询多个对象的关联对象的时候，可以采用批量抓取进行优化。当然这个就不是特别的重要了。

如果来实现批量的抓取效果，可以通过配置 batch-size 来完成。来看下下面的效果：

【查询客户批量抓取联系人】

```

@Test
// 查询客户的时候批量抓取联系人
// 在 Customer.hbm.xml 中<set>集合上配置 batch-size="4"
public void demo1() {

```




```

Session session = HibernateUtils.getCurrentSession();
Transaction transaction = session.beginTransaction();

List<Customer> list = session.createQuery("from Customer").list();
for (Customer customer : list) {
    System.out.println("客户名称:" + customer.getCust_name());
    for (LinkMan linkMan : customer.getLinkMans()) {
        System.out.println("客户对应的联系人的名称:" + linkMan.getLkm_name());
    }
}
transaction.commit();
}

```

在没有设置 batch-size 之前运行以上代码，会出现如下的效果：

```

15:59:50,978 INFO QueryTranslatorFactoryInitiator:47 - HH000397: Using ASTQueryTranslatorFactory
Hibernate:
select
  customer0_.cust_id as cust_id1_0_,
  customer0_.cust_name as cust_name2_0_,
  customer0_.cust_source as cust_sou3_0_,
  customer0_.cust_industry as cust_ind4_0_,
  customer0_.cust_level as cust_level5_0_,
  customer0_.cust_phone as cust_phone6_0_,
  customer0_.cust_mobile as cust_mob7_0_
from
  cst_customer customer0_
客户名称:张总
Hibernate:
select
  linkmans0_.lkm_cust_id as lkm_cust10_1_0_,
  linkmans0_.lkm_id as lkm_id1_1_0_,
  linkmans0_.lkm_id as lkm_id1_1_1_,
  linkmans0_.lkm_name as lkm_name2_1_1_,
  linkmans0_.lkm_gender as lkm_gender3_1_1_,
  linkmans0_.lkm_phone as lkm_phone4_1_1_,
  linkmans0_.lkm_mobile as lkm_mobile5_1_1_,
  linkmans0_.lkm_email as lkm_email6_1_1_,
  linkmans0_.lkm_qq as lkm_qq7_1_1_,
  linkmans0_.lkm_position as lkm_position8_1_1_,
  linkmans0_.lkm_memo as lkm_memo9_1_1_,
  linkmans0_.lkm_cust_id as lkm_cust10_1_1_
from
  cst_linkman linkmans0_
where
  linkmans0_.lkm_cust_id=?
客户对应的联系人的名称:王助理
客户对应的联系人的名称:李秘书
客户名称:刘总
Hibernate:
select
  linkmans0_.lkm_cust_id as lkm_cust10_1_0_,
  linkmans0_.lkm_id as lkm_id1_1_0_,
  linkmans0_.lkm_id as lkm_id1_1_1_,
  linkmans0_.lkm_name as lkm_name2_1_1_,
  linkmans0_.lkm_gender as lkm_gender3_1_1_,
  linkmans0_.lkm_phone as lkm_phone4_1_1_,
  linkmans0_.lkm_mobile as lkm_mobile5_1_1_,
  linkmans0_.lkm_email as lkm_email6_1_1_,
  linkmans0_.lkm_qq as lkm_qq7_1_1_,
  linkmans0_.lkm_position as lkm_position8_1_1_,
  linkmans0_.lkm_memo as lkm_memo9_1_1_,
  linkmans0_.lkm_cust_id as lkm_cust10_1_1_
from
  cst_linkman linkmans0_
where
  linkmans0_.lkm_cust_id=?
客户对应的联系人的名称:张秘书

```

目前数据库中有两个客户，那么会发现执行代码会发生三条 SQL 语句。那么我们能不能发生一条 SQL 直接将两个客户的关联的联系人一起查询出来呢，我们可以在<set>上配置 batch-size 实现优化效果。



```
<!-- 配置关联关系映射: -->
<!-- 配置一个set标签:代表一个集合 name属性: 集合的属性名称 -->
<set name="LinkMans" batch-size="2">
    <!-- 配置一个key标签 column属性:多的一方的外键的名称 -->
    <key column="lkm_cust_id"/>
    <!-- 配置one-to-many class对用多的一方的类的全路径 -->
    <one-to-many class="cn.itcast.hibernate.demo1.LinkMan"/>
</set>
```

配置完了 batch-size 以后，再次运行以上的代码会发现：

16:04:10,477 INFO QueryTranslatorFactoryInitiator:47 - HH000397: Using ASTQueryTranslatorFactory

```
Hibernate:
select
  customer0_.cust_id as cust_id1_0_,
  customer0_.cust_name as cust_nam2_0_,
  customer0_.cust_source as cust_sou3_0_,
  customer0_.cust_industry as cust_ind4_0_,
  customer0_.cust_level as cust_lev5_0_,
  customer0_.cust_phone as cust_pho6_0_,
  customer0_.cust_mobile as cust_mob7_0_
from
  cst_customer customer0_
客户名称:张总
Hibernate:
select
  linkmans0_.lkm_cust_id as lkm_cus10_1_1_,
  linkmans0_.lkm_id as lkm_id1_1_1_,
  linkmans0_.lkm_id as lkm_id1_1_0_,
  linkmans0_.lkm_name as lkm_name2_1_0_,
  linkmans0_.lkm_gender as lkm_gend3_1_0_,
  linkmans0_.lkm_phone as lkm_phon4_1_0_,
  linkmans0_.lkm_mobile as lkm_mobi5_1_0_,
  linkmans0_.lkm_email as lkm_ema6_1_0_,
  linkmans0_.lkm_qq as lkm_qq7_1_0_,
  linkmans0_.lkm_position as lkm_posi8_1_0_,
  linkmans0_.lkm_memo as lkm_memo9_1_0_,
  linkmans0_.lkm_cust_id as lkm_cus10_1_0_
from
  cst_linkman linkmans0_
where
  linkmans0_.lkm_cust_id in (
    ?, ?
  )
客户对应的联系人的名称:王助理
客户对应的联系人的名称:李秘书
客户名称:刘总
客户对应的联系人的名称:张秘书
```

这时会发现 SQL 语句发生了变化，当然数据量越大，效果越明显。这是在查询所有客户的时候，批量抓取联系人，那么如果我们要实现查询多个联系人的时候，批量抓取联系人所对应的客户对象呢？这时同样需要配置 batch-size。但是不是在<many-to-one>上配置，还是在客户一端的<class>上配置 batch-size 即可。

【查询联系人批量抓取客户】

```
@Test
// 查询联系人的时候批量抓取客户
// 在 Customer.hbm.xml 中<class>上配置 batch-size="2"
public void demo2() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    List<LinkMan> list = session.createQuery("from LinkMan").list();
    for (LinkMan linkMan : list) {
        System.out.println(linkMan.getLkm_name());
        System.out.println(linkMan.getCustomer().getCust_name());
    }
}
```



```
transaction.commit();
}
```

如果没有配置 batch-size 执行上述代码会得到如下效果：

16:09:00,216 INFO QueryTranslatorFactoryInitiator:47 - HHH000397: Using ASTQueryTranslatorFactory

Hibernate:

```
select
  linkman0_.lkm_id as lkm_id1_1_,
  linkman0_.lkm_name as lkm_name2_1_,
  linkman0_.lkm_gender as lkm_gender3_1_,
  linkman0_.lkm_phone as lkm_phone4_1_,
  linkman0_.lkm_mobile as lkm_mobile5_1_,
  linkman0_.lkm_email as lkm_email6_1_,
  linkman0_.lkm_qq as lkm_qq7_1_,
  linkman0_.lkm_position as lkm_position8_1_,
  linkman0_.lkm_memo as lkm_memo9_1_,
  linkman0_.lkm_cust_id as lkm_cust10_1_
from
  cst_linkman linkman0_
```

李秘书

Hibernate:

```
select
  customer0_.cust_id as cust_id1_0_0_,
  customer0_.cust_name as cust_name2_0_0_,
  customer0_.cust_source as cust_source3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_level5_0_0_,
  customer0_.cust_phone as cust_phone6_0_0_,
  customer0_.cust_mobile as cust_mobile7_0_0_
from
  cst_customer customer0_
where
  customer0_.cust_id=?
```

张总

王助理

张总

张秘书

Hibernate:

```
select
  customer0_.cust_id as cust_id1_0_0_,
  customer0_.cust_name as cust_name2_0_0_,
  customer0_.cust_source as cust_source3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_level5_0_0_,
  customer0_.cust_phone as cust_phone6_0_0_,
  customer0_.cust_mobile as cust_mobile7_0_0_
from
  cst_customer customer0_
where
  customer0_.cust_id=?
```

刘总

如果配置了 batch-size 以后，再次执行上述代码得到效果如下：

注意：这时候要在客户一端的映射文件中的 class 标签上配置 batch-size。

```
<class name="cn.itcast.hibernate.demo1.Customer" table="cst_customer" batch-size="2">
```

效果如下：



16:11:07,191 INFO QueryTranslatorFactoryInitiator:47 - HH000397: Using ASTQueryTranslatorFactory

Hibernate:

```
select
  linkman0_.lkm_id as lkm_id1_1_,
  linkman0_.lkm_name as lkm_name2_1_,
  linkman0_.lkm_gender as lkm_gend3_1_,
  linkman0_.lkm_phone as lkm_phon4_1_,
  linkman0_.lkm_mobile as lkm_mobi5_1_,
  linkman0_.lkm_email as lkm_email6_1_,
  linkman0_.lkm_qq as lkm_qq7_1_,
  linkman0_.lkm_position as lkm_posi8_1_,
  linkman0_.lkm_memo as lkm_memo9_1_,
  linkman0_.lkm_cust_id as lkm_cust10_1_
from
  cst_linkman linkman0_
```

李秘书

Hibernate:

```
select
  customer0_.cust_id as cust_id1_0_0_,
  customer0_.cust_name as cust_name2_0_0_,
  customer0_.cust_source as cust_sou3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_level5_0_0_,
  customer0_.cust_phone as cust_phone6_0_0_,
  customer0_.cust_mobile as cust_mob7_0_0_
from
  cst_customer customer0_
```

```
where
  customer0_.cust_id in (
    ?, ?
  )
```

张总

王助理

张总

张秘书

刘总

这时候可以看到，语句和之前就已经发送了变化了。这些优化都是 Hibernate 提升自身性能的手段。